



**TEKNOLOJİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**

Afet Bilgilendirme ve Acil Konum Paylaşımı Yapan Web Uygulaması

Proje Ekibi

Berat Erdoğdu - 171421010
Raşit Bera Topal - 170422516
Abdurrahman Çoban - 170422851

Danışman

Prof. Dr. Ali Buldu

İÇİNDEKİLER

ÖZET

1. BÖLÜM: GİRİŞ

- 1.1. Problemin Tanımı ve Arka Plan
- 1.2. Projenin Amacı ve Kapsamı
- 1.3. Literatür Taraması ve Mevcut Sistemlerin İncelenmesi
- 1.4. Projenin Hedef Kitle ve Sağlayacağı Faydalar

2. BÖLÜM: SİSTEM GEREKSİNİMLERİ VE ANALİZİ

- 2.1. Fonksiyonel Gereksinimler
 - 2.1.1. Rol Bazlı Yetkilendirme ve Kullanıcı Yönetimi
 - 2.1.2. İhbar ve Hasar Tespit Modülü
 - 2.1.3. AFAD Entegrasyonu ve Veri Senkronizasyonu
- 2.2. Fonksiyonel Olmayan Gereksinimler
 - 2.2.1. Güvenlik Gereksinimleri
 - 2.2.2. Performans ve Ölçeklenebilirlik
 - 2.2.3. PWA ve Çevrimdışı (Offline) Çalışabilirlik
- 2.3. Kullanıcı Senaryoları (Use-Case Analizleri)
- 2.4. Sistemin Görselleştirilmesi: Use-Case Diyagramı
- 2.5. Sistem Sıralama (Sequence) Diyagramı Önerisi

3. BÖLÜM: KULLANILAN TEKNOLOJİLER VE GELİŞTİRME ORTAMI

- 3.1. Backend Teknolojileri
 - 3.1.1. Java 17 ve Spring Boot 3.x
 - 3.1.2. Katmanlı Mimari ve İş Mantığı
 - 3.1.3. Güvenlik ve Yetkilendirme (Spring Security & JWT)
- 3.2. Frontend Teknolojileri
 - 3.2.1. React 18, Vite ve TypeScript
 - 3.2.2. Durum Yönetimi: Zustand ve React Query .
 - 3.2.3. PWA (Progressive Web App) ve Offline Kapasite
- 3.3. Veritabanı ve Veri Yönetimi
 - 3.3.1. PostgreSQL ve Mimari Avantajları
 - 3.3.2. Flyway ile Veritabanı Versiyonlama
- 3.4. Harita ve Yapay Zeka Servisleri
 - 3.4.1. Leaflet ve React Leaflet
 - 3.4.2. Anthropic Claude Vision API Entegrasyonu

4. BÖLÜM: SİSTEM MİMARİSİ VE TASARIM

- 4.1. Katmanlı Mimari (Layered Architecture)
- 4.2. Veritabanı İlişkisel Tasarımı
- 4.3. Güvenlik Mimarisi
- 4.4. Dış Servis Entegrasyonları

5. BÖLÜM: UYGULAMA ARAYÜZÜ VE İŞLEYİŞ

- 5.1. Kullanıcı Kayıt ve Rol Onay Süreçleri
- 5.2. Aktif Afet Durumu ve Canlı Harita Ekranları
- 5.3. Yapay Zeka Destekli Hasar İhbar Süreci
- 5.4. Gönüllü ve Kaynak Yönetim Ekranları

6. BÖLÜM: TEST SÜREÇLERİ VE PERFORMANS ANALİZİ

- 6.1. Birim (Unit) ve Entegrasyon Test Stratejisi
- 6.2. API Yanıt Süreleri ve Postman Test Sonuçları
- 6.3. PostgreSQL Veritabanı Yük Testleri ve İndeksleme Performansı
- 6.4. Yapay Zeka Görsel Analiz Süreleri ve Optimizasyonları
- 6.5. PWA ve Çevrimdışı (Offline) Dayanıklılık Testleri

7. BÖLÜM: SONUÇ, DEĞERLENDİRME VE GELECEK ÇALIŞMALAR

- 7.1. Elde Edilen Başarılar ve Hedeflere Ulaşım
- 7.2. Karşılaşılan Zorluklar ve Üretilen Çözümler
- 7.3. Gelecekte Eklenebilecek Özellikler

8. BÖLÜM: KAYNAKÇA

9. BÖLÜM: EKLER

ÖZET

Doğal afetlerin yıkıcı etkileri ve beklenmedik sıklığı, kriz yönetimi ile acil durum müdahale süreçlerinin dijital teknolojilerle entegre edilmesini birincil öncelik haline getirmiştir. Özellikle İstanbul gibi yüksek deprem riski taşıyan bölgelerde, afet anında yaşanan iletişim kopuklukları ve koordinasyon eksikliği kritik gecikmelere ve can kayıplarına yol açabilmektedir. Bu çalışma kapsamında geliştirilen "**Afet Koordinasyon Platformu (AKP)**", vatandaşların, gönüllülerin ve yetkili kurumların tek bir çatı altında toplanmasını sağlayan, modüler ve yüksek erişilebilirlikli bir dijital koordinasyon altyapısı sunmaktadır.

Projenin teknik mimarisi; kurumsal düzeyde performans sunan **Java 17 ve Spring Boot 3** tabanlı bir arka uç (backend), modern **React 18 ve TypeScript** tabanlı bir ön uç (frontend) ve **PostgreSQL** veritabanı üzerine inşa edilmiştir. Sistem, sorumlulukların ayrıştırıldığı "**Üç Katmanlı (Three-Tier) Dağıtık Mimari**" prensiplerini benimsemekte ve **Flyway** migration sistemi ile sürüm kontrollü bir veritabanı yönetimi sağlamaktadır.

Platformun en önemli özgün değerlerinden birini, **Anthropic Claude Vision API** kullanılarak geliştirilen yapay zeka destekli bina hasar ön değerlendirme modülü oluşturmaktadır. Bu modül sayesinde sahadan yüklenen fotoğraflar analiz edilerek, uzman ekipler için hızlı bir önceliklendirme verisi üretilmektedir. Ayrıca, afet anındaki internet kesintilerine karşı **Progressive Web App (PWA)** mimarisi ve **IndexedDB** tabanlı çevrimdışı (offline) çalışma desteği ile temel işlevlerin sürekliliği garanti altına alınmıştır.

Sistem içerisinde; gerçek zamanlı **AFAD deprem verisi entegrasyonu**, Leaflet kütüphanesi ile zenginleştirilmiş **harita tabanlı görselleştirme**, hiyerarşik rol bazlı yetkilendirme (RBAC) ve kaynak/gönüllü yönetimi gibi operasyonel modüller yer almaktadır. Kullanıcıların konum bilgisine dayalı olarak en yakın güvenli toplanma alanlarına yönlendirilmesi ve SMS/OTP tabanlı iki aşamalı giriş mekanizmaları ile platformun güvenliği ve kapsayıcılığı en üst düzeye çıkarılmıştır. Sonuç olarak bu proje, afetlerin yıkıcı etkilerini minimize etmeyi amaçlayan, katılımcı ve teknolojik bir karar destek sistemi sunarak toplumun afet dayanıklılığını güçlendirmeyi hedeflemektedir.

Anahtar Kelimeler: Afet Yönetimi, Spring Boot, React, Yapay Zeka, PWA, Hasar Tespiti, AFAD Entegrasyonu.

1. BÖLÜM: GİRİŞ

Doğal afetler ve acil durumlar, modern toplumların karşı karşıya kaldığı en büyük risk unsurlarından biri olarak kabul edilmektedir. Bireylerin yaşam güvenliğini, kamu düzenini, ekonomik istikrarı ve kritik altyapı sistemlerini doğrudan tehdit eden bu olaylar; ani gelişimleri ve geniş ölçekli etkileri nedeniyle "proaktif" bir yönetim anlayışını zorunlu kılmaktadır. Afet anlarında yaşanan kaos, iletişim altyapılarının çökmesi ve bilgiye hızlı erişimdeki aksaklıklar, yardım süreçlerinde telafisi olmayan gecikmelere ve can kayıplarına yol açmaktadır. Bu bağlamda, dijital teknolojilerin ve modern yazılım mühendisliği çözümlerinin afet yönetim süreçlerine entegre edilmesi, hem devlet kurumları hem de sivil toplum kuruluşları için artık bir seçenek değil, yaşamsal bir zorunluluk haline gelmiştir.

Gelişen konum tabanlı servisler (LBS), yapay zeka algoritmaları ve dayanıklı mobil iletişim mimarileri; afetzedelerin yerinin tespit edilmesini, hasar tespit süreçlerinin hızlandırılmasını ve gönüllü kaynaklarının optimize edilmesini mümkün kılmaktadır. Bu çalışma kapsamında geliştirilen "**Afet Koordinasyon Platformu (AKP)**", mevcut sistemlerin sunduğu tek yönlü bilgi paylaşımının ötesine geçerek; vatandaş, gönüllü ve kamu otoriteleri arasında çift yönlü, dinamik ve katılımcı bir ekosistem sunmayı hedefleyen bir üst yapı projesidir.

1.1. Problemin Tanımı ve Arka Plan

Türkiye, jeolojik yapısı ve tektonik konumu itibarıyla dünyanın en aktif deprem kuşaklarından birinde yer almaktadır. Özellikle beklenen büyük İstanbul depremi gibi senaryolar, mega kentlerin afet hazırlık süreçlerini ulusal bir güvenlik sorunu haline getirmiştir. Geçmişte tecrübe edilen 1999 Marmara depremi ve yakın dönemde yaşanan büyük ölçekli sarsıntılar, afet yönetiminde sadece fiziksel müdahalenin değil, "bilgi koordinasyonunun" da ne kadar kritik olduğunu göstermiştir.

Mevcut afet yönetim sistemleri incelendiğinde, kriz anlarında operasyonel verimliliği düşüren temel sistematik sorunlar şu şekilde tespit edilmiştir:

- **Dağınık Gönüllü Yönetimi:** Afet anında bölgeye akın eden binlerce gönüllünün yetkinliklerine (arama-kurtarma, lojistik, tıbbi destek vb.) göre sınıflandırılmaması ve doğru ihtiyaç noktalarına yönlendirilememesi büyük bir kaynak israfına yol açmaktadır.
- **Hasar Tespitindeki Yavaşlık:** Geleneksel yöntemlerle yapılan bina hasar tespit çalışmaları, uzman ekiplerin fiziksel olarak her binayı gezmesine dayanmaktadır. Bu süreç, krizin ilk kritik saatlerinde yardımın önceliklendirilmesini imkansız hale getirmektedir.
- **Merkezi Olmayan Veri Akışı:** Kaynak taleplerinin (su, gıda, çadır vb.) mahalle düzeyinden ilçe ve merkez düzeyine aktarılmasında yaşanan kopukluklar, yardımların belirli bölgelerde yığılmasına, bazı bölgelerin ise mahrum kalmasına neden olmaktadır.

- **İletişim Altyapısının Kırılganlığı:** Mevcut dijital araçların büyük çoğunluğu kesintisiz internet erişimine ihtiyaç duymaktadır. Ancak afet anlarında baz istasyonlarının çökmesi veya bant genişliğinin daralması, bu sistemleri tamamen işlevsiz bırakmaktadır.
- **Resmi Veri Entegrasyonu Eksikliği:** Sahada görev yapan ekiplerin AFAD gibi resmi kurumlardan gelen anlık deprem ve risk verilerine operasyonel sistemler üzerinden doğrudan erişememesi, karar alma mekanizmalarını yavaşlatmaktadır.

1.2. Projenin Amacı ve Kapsamı

"Afet Bilgilendirme ve Acil Konum Paylaşımı Yapan Web Uygulaması" projesinin temel amacı; afet öncesi, afet anı ve afet sonrasını kapsayan modüler, yüksek erişilebilirlikli ve yapay zeka destekli bir koordinasyon altyapısı kurmaktır. Platform, sadece teknik bir yazılım değil, aynı zamanda farklı paydaşların (Admin, İlçe Koordinatörü, Mahalle Koordinatörü ve Gönüllü) hiyerarşik bir düzende çalışabildiği bir karar destek sistemi olarak kurgulanmıştır.

Projenin kapsamı, modern yazılım mimarileriyle desteklenen şu ana bileşenlerden oluşmaktadır:

- **Yapay Zeka Destekli Hasar Ön Değerlendirme:** Sahadan vatandaşlar veya ekipler tarafından yüklenen bina fotoğraflarının **Anthropic Claude Vision API** kullanılarak analiz edilmesi ve hasar seviyesinin (hafif, orta, ağır, yıkık) saniyeler içinde tahmin edilmesi.
- **PWA (Progressive Web App) ve Çevrimdışı Destek:** İnternet kesintilerine karşı sistemin temel işlevlerinin (form doldurma, harita görüntüleme) çalışmaya devam etmesi ve bağlantı geldiğinde verilerin otomatik senkronize edilmesi.
- **Konum Tabanlı Acil Durum Yönetimi:** Kullanıcıların koordinat verilerini paylaşarak en yakın güvenli toplanma alanlarına yönlendirilmesi ve yardım taleplerinin harita üzerinde GeoJSON formatında görselleştirilmesi.
- **Resmi AFAD Entegrasyonu:** AFAD API servisleri ile sağlanan bağlantı sayesinde, deprem verilerinin gerçek zamanlı olarak sisteme çekilmesi ve eşik değerlerin üzerindeki sarsıntılarda otomatik bildirim mekanizmalarının tetiklenmesi.
- **Gönüllü ve Ekip Organizasyonu:** Gönüllülerin sertifika ve uzmanlık belgelerinin dijital ortamda doğrulanması, ekiplerin mahalle bazlı görevlendirilmesi ve operasyonel risk skorlarının hesaplanması.
- **SMS ve Çok Kanallı İletişim:** İnternet erişimi olmayan kullanıcılar için SMS OTP (Tek Kullanımlık Şifre) tabanlı giriş ve hazır durum mesajları (Güvendeyim, Yardıma İhtiyacım Var vb.) ile iletişim sürekliliğinin sağlanması.

1.3. Literatür Taraması ve Mevcut Sistemlerin İncelenmesi

Afet yönetiminde bilgi sistemlerinin kullanımı üzerine yapılan akademik çalışmalar, konum tabanlı servislerin ve gerçek zamanlı veri analizinin müdahale sürelerini %40'a varan oranlarda kısalttığını ortaya koymaktadır. Uluslararası literatürde FEMA (ABD) ve J-ALERT (Japonya) gibi sistemler, merkezi uyarı mekanizmalarında başarılı örnekler olarak gösterilse de; bu sistemlerin yerel gönüllü katılımı ve mahalle düzeyinde hasar tespiti konularındaki esnekliği hala tartışma konusudur.

Türkiye'deki mevcut dijital çözümler incelendiğinde; sistemlerin çoğunlukla "tek yönlü" bilgi aktarımı (sadece uyarı gönderme) üzerine odaklandığı tespit edilmiştir. Vatandaşların aktif olarak veri girişi yapabildiği (katılımcı afet yönetimi) ve yapay zeka ile sahadan gelen görsel verilerin anlık işlendiği entegre bir platform eksikliği mevcuttur.

Bu proje, literatürdeki ve pratikteki bu eksiklikleri gidermek adına;

1. **Spring Boot 3** ve **Java 17** tabanlı kurumsal düzeyde güvenli bir backend,
2. **React 18** ve **TypeScript** ile geliştirilmiş kullanıcı dostu bir frontend,
3. **PostgreSQL** ve **JSONB** veri yapılarıyla zenginleştirilmiş coğrafi veri yönetimi,
4. **Docker** ve **Flyway** gibi modern dağıtım ve sürümleme teknolojilerini bir araya getirerek özgün bir değer sunmaktadır.

1.4. Projenin Hedef Kitlesi ve Sağlayacağı Faydalar

Afet Koordinasyon Platformu, dört ana kullanıcı grubunu hedefleyen kapsayıcı bir yapıya sahiptir:

- **Afetzedeler ve Vatandaşlar:** Hızlı bir şekilde konum paylaşabilir, yardım talebi oluşturabilir, "güvendeyim" mesajıyla yakınlarına ulaşabilir ve en yakın toplanma alanını harita üzerinde görebilirler.
- **Gönüllüler:** Yetkinliklerine göre uzman ekiplere (arama-kurtarma, lojistik vb.) katılabilir, kendilerine atanan görevleri takip edebilir ve sahada aktif rol alabilirler.
- **Koordinatörler (İlçe ve Mahalle):** Kendi sorumluluk alanlarındaki hasar bildirimlerini onaylayabilir, kaynak taleplerini yönetebilir ve bölgenin risk skorunu anlık olarak izleyebilirler.
- **Sistem Yöneticileri (Admin):** Tüm Türkiye genelindeki koordinasyonu sağlar, AFAD veri senkronizasyonunu yönetir ve simülasyonlar başlatarak sistemin hazırlık seviyesini test ederler.

Sistemin Sağlayacağı Temel Faydalar:

- **Hayat Kurtaran Hız:** Yapay zeka ve dijital hasar tespiti sayesinde, müdahale ekipleri en çok ihtiyaç duyulan (çökmüş veya ağır hasarlı) binalara öncelikli olarak yönlendirilir.
- **Kaynak Optimizasyonu:** Su, gıda ve tıbbi malzeme gibi kritik kaynaklar, harita tabanlı talep modülü sayesinde "ihtiyaç duyulan miktarda ve doğru yere" ulaştırılır.
- **Erişilebilirlik:** PWA teknolojisi ve SMS entegrasyonu ile internetin en kısıtlı olduğu anlarda bile sistemin devre dışı kalması önlenir.
- **Karar Destek Kapasitesi:** Toplanan tüm veriler (audit loglar, görev kayıtları, hasar puanları) analiz edilerek, gelecekteki afetlere yönelik stratejik planlama yapılmasına imkan tanır.

Bu çalışma, teknolojik imkanları insan odaklı bir afet yönetim vizyonu ile birleştirilerek; afetlerin yıkıcı etkilerini minimize etmeyi ve toplumun afetlere karşı dayanıklılığını (resilience) dijital bir kalkanla güçlendirmeyi amaçlamaktadır.

2. BÖLÜM: SİSTEM GEREKSİNİMLERİ VE ANALİZİ

Afet Koordinasyon Platformu'nun (AKP) geliştirilme sürecinde, sistemin karmaşık afet senaryoları altında nasıl davranması gerektiğini belirleyen gereksinimler; fonksiyonel (işlevsel) ve fonksiyonel olmayan (kalite odaklı) olmak üzere iki ana kategoride analiz edilmiştir. Bu analiz, platformun sadece teknik bir yazılım değil, kriz anlarında hayat kurtaran bir karar destek mekanizması olarak kurgulanmasını sağlamaktadır.

2.1. Fonksiyonel Gereksinimler

Sistemin fonksiyonel gereksinimleri, farklı kullanıcı gruplarının platform üzerindeki yetki ve sorumluluklarına göre modüler bir hiyerarşi içerisinde yapılandırılmıştır. Platform, "Admin", "İlçe Koordinatörü", "Mahalle Koordinatörü" ve "Gönüllü" olmak üzere dört ana kullanıcı rolünü desteklemektedir.

2.1.1. Rol Bazlı Yetkilendirme ve Kullanıcı Yönetimi

Sistem, hiyerarşik bir yetki akışına sahiptir ve her işlem backend düzeyinde Spring Security altyapısıyla denetlenmektedir.

- **Admin (Sistem Yöneticisi):** Platformun en üst düzey yetkilisidir. Kullanıcı yönetimi, yeni koordinatör atamaları, sistem bakımı ve kritik AFAD veri senkronizasyon süreçlerini yönetir. Ayrıca afet simülasyonları başlatarak sistemin hazırlık seviyesini ölçmekten sorumludur.

- **İlçe Koordinatörü:** Sorumlu olduğu ilçe sınırları içerisindeki tüm mahallelerin, koordinasyon merkezlerinin ve kaynak taleplerinin yönetiminden sorumludur. İlçedeki hasar tespit süreçlerini onaylama yetkisine sahiptir.
- **Mahalle Koordinatörü:** Sahadan gelen anlık hasar bildirimlerini takip eder, gönüllülerin görevlendirilmesini sağlar ve mahalle düzeyindeki kaynak ihtiyaçlarını (su, gıda, tıbbi destek) sisteme girer.
- **Gönüllü (Vatandaş):** Sistemin en geniş kullanıcı kitlesidir. Profil oluşturma, sertifika ve belgelerini yükleyerek ekiplere dahil olma, konum bilgisi paylaşma ve sahadan hasar bildirimi yapma yetkilerine sahiptir.



2.1.2. İhbar ve Hasar Tespit Modülü

Bu modül, afet anında sahadan gelen ham verilerin anlamlı bilgilere dönüştürülmesini sağlar.

- **Acil Durum Bildirimi:** Kullanıcılar tek bir tuşla enlem ve boylam bilgilerini içeren acil yardım talepleri oluşturabilmektedir.
- **AI Destekli Ön Değerlendirme:** Sahadan yüklenen bina fotoğrafları, Anthropic Claude Vision API kullanılarak analiz edilir. Yapay zeka; binanın hasar durumunu (hafif, orta, ağır, yıkık) saniyeler içinde yorumlayarak bir güven skoru (confidence score) üretir.
- **Doğrulama Akışı:** AI tarafından yapılan ön değerlendirme, sırasıyla saha ekipleri ve koordinatörler tarafından doğrulanarak "Onaylandı" statüsüne getirilir.

2.1.3. AFAD Entegrasyonu ve Veri Senkronizasyonu

Sistem, AFAD deprem veri API'sine doğrudan bağlanarak Türkiye genelindeki sarsıntılarını gerçek zamanlıya yakın şekilde takip eder. Belirlenen büyüklük eşiğinin (örn. 4.0 ve üzeri) üstündeki depremler için sistem otomatik olarak koordinatörlere bildirim gönderir ve bölgedeki gönüllüleri teyakkuza geçirir.

2.2. Fonksiyonel Olmayan Gereksinimler

Sistemin başarısı sadece ne yaptığıyla değil, zorlu koşullar altında ne kadar güvenilir ve performanslı çalıştığıyla ölçülmektedir.

2.2.1. Güvenlik Gereksinimleri

Afet anında paylaşılan konum ve kişisel verilerin korunması en yüksek önceliğe sahiptir.

- **Kimlik Doğrulama:** Sistem, JWT (JSON Web Token) tabanlı bir kimlik doğrulama mimarisi kullanır. Şifreler BCrypt algoritması ile hashlenerek saklanır.
- **İki Aşamalı Giriş (2FA):** Kritik kullanıcılar için SMS OTP (Tek Kullanımlık Şifre) desteği sunularak hesap güvenliği artırılmıştır.
- **Denetim Kayıtları (Audit Log):** Sistemdeki her kritik işlem (rol değişikliği, yardım onayı vb.) kimin tarafından ne zaman yapıldığına dair tarihsel bir iz kaydıyla saklanır.

2.2.2. Performans ve Ölçeklenebilirlik

- **Yüksek Eş Zamanlılık:** Sistem, binlerce kullanıcının aynı anda veri girişi yapabileceği afet anlarındaki yoğun yükü yönetebilmek için Spring Boot'un kurumsal düzeydeki API performansından yararlanır.
- **Veritabanı Optimizasyonu:** PostgreSQL üzerinde UUID primary key ve coğrafi veriler için JSONB veri tipleri kullanılarak sorgu performansları optimize edilmiştir.
- **Cevap Süreleri:** Kritik API çağrılarının (konum paylaşımı gibi) düşük gecikme süreleriyle sonuçlanması hedeflenmiştir.

2.2.3. PWA ve Çevrimdışı (Offline) Çalışabilirlik

İletişim altyapısının çöktüğü senaryolar için platform Progressive Web App (PWA) mimarisini benimsemiştir.

- **Statik Önbellekleme:** Uygulamanın arayüz bileşenleri ve harita karoları (map tiles) cihaz hafızasına önbelleğe alınır.

- **Offline Kuyruk (Queue):** İnternet bağlantısı kesildiğinde yapılan işlemler (hasar raporu, yardım talebi vb.) IndexedDB üzerinde bir kuyruğa alınır. Bağlantı geri geldiğinde bu veriler otomatik olarak sunucuya senkronize edilir.
- **Erişilebilirlik:** İnterneti olmayan kullanıcıların SMS kodlarıyla sisteme giriş yapabilmesi sağlanmıştır.

2.3. Kullanıcı Senaryoları (Use-Case Analizleri)

Sistemin işleyişini somutlaştırmak amacıyla belirlenen temel kullanım senaryoları aşağıda adım adım açıklanmıştır.

2.3.1. Senaryo 1: Afetzedede Tarafından Yardım Talebi Oluşturma

Bu senaryo, sistemin en temel varlık nedenini temsil eder.

1. **Tetikleyici:** Afetzedede uygulamayı açar ve "Yardım İste" butonuna basar.
2. **Veri Girişi:** Sistem kullanıcıdan ihtiyaç türünü (Sağlık, Gıda, Enkaz Altında vb.) seçmesini ister.
3. **Konum Tespiti:** PWA üzerinden cihazın GPS koordinatları otomatik olarak alınır.
4. **İşlem:** Talep backend'e iletilir, GeoJSON formatında veritabanına kaydedilir ve ilgili mahalle koordinatörünün ekranına "Acil" koduyla düşer.
5. **Sonuç:** Kullanıcıya talebinin alındığı ve en yakın ekibin yönlendirildiği bilgisi sunulur.

2.3.2. Senaryo 2: Yapay Zeka Destekli Hasar Tespiti

1. **Tetikleyici:** Bir gönüllü veya saha personeli hasarlı bir binanın önündedir.
2. **Veri Girişi:** Uygulama üzerinden binanın dış cephesini gösteren en az bir fotoğraf yükler.
3. **AI Analizi:** Sistem, fotoğrafı Claude Vision API'ye gönderir. Yapay zeka, binadaki yapısal çatlakları analiz ederek bir "Hasar Tahmin Raporu" oluşturur.
4. **Onay:** Bu rapor mahalle koordinatörü tarafından incelenir. Koordinatör, AI sonucuna dayanarak binanın tahliyesine karar verebilir veya saha ekibini fiziksel inceleme için atar.

2.3.3. Senaryo 3: Gönüllü Ekip ve Görev Yönetimi

1. **Aksiyon:** Sisteme yeni kayıt olan bir gönüllü "Arama-Kurtarma" ekibine katılmak ister.
2. **Belge Kontrolü:** Sistem, bu ekip için zorunlu olan sertifikayı yüklemesini ister.

3. **Onay Süreci:** Yüklenen belge Admin veya İlçe Koordinatörü tarafından dijital olarak doğrulanır.
4. **Görevlendirme:** Sertifikası onaylanan gönüllü, harita üzerinde kendi mahallesindeki aktif görevleri (örn. "X binasında lojistik destek") görmeye başlar ve "Görevi Üstlen" diyerek sürece dahil olur.

2.3.4. Senaryo 4: SMS Tabanlı Durum Bildirimi (İnternetsiz Durum)

1. **Senaryo:** Bölgede internet altyapısı tamamen çökmüştür.
2. **Aksiyon:** Kullanıcı sistemin belirlediği kısa numaraya "GÜVENDYİM" yazarak SMS gönderir.
3. **İşleme:** SMS servis sağlayıcısı (İleti Merkezi) bu mesajı backend'e iletir.
4. **Kayıt:** Sistem, kullanıcının profilindeki durumunu otomatik olarak "Güvende" olarak günceller ve kullanıcının önceden tanımladığı "Acil Durum Kişileri"ne bilgi gönderir.

3. BÖLÜM: KULLANILAN TEKNOLOJİLER VE GELİŞTİRME ORTAMI

Afet Koordinasyon Platformu'nun (AKP) teknik altyapısı; afet anlarındaki yüksek veri trafiğini yönetebilecek, internet kesintilerine karşı dayanıklılık sunacak ve yapay zeka gibi modern çözümleri bünyesinde barındıracak şekilde kurgulanmıştır. Projenin geliştirilme sürecinde, sürdürülebilirlik ve performans kriterleri göz önünde bulundurularak "Üç Katmanlı (Three-Tier) Dağıtık Mimari" benimsenmiş; backend, frontend ve veritabanı katmanları modern yazılım pratikleriyle inşa edilmiştir.

3.1. Backend Teknolojileri

Sistemin kalbini oluşturan arka uç (backend) mimarisi, Java ekosisteminin en güncel ve güvenilir bileşenleri kullanılarak, kurumsal düzeyde bir RESTful API yapısı sunacak şekilde geliştirilmiştir.

3.1.1. Java 17 ve Spring Boot 3.x

Projenin temel programlama dili olarak **Java 17 (LTS)** tercih edilmiştir. Java 17'nin sunduğu "Record" tipi veriler, geliştirilmiş "Switch" ifadeleri ve performans iyileştirmeleri, sistemin daha temiz ve tip güvenli bir kod yapısına sahip olmasını sağlamıştır.

Uygulama çatısı (framework) olarak **Spring Boot 3.x** kullanılmıştır. Spring Boot, bağımlılık yönetimi ve otomatik yapılandırma özellikleri sayesinde geliştirme sürecini hızlandırırken; mikroservis mimariye geçişe uygun modüler yapısıyla sistemin gelecekteki ölçeklenebilirliğini garanti altına almaktadır.

3.1.2. Katmanlı Mimari (Layered Architecture) ve İş Mantığı

Backend yapısı, sorumlulukların net bir şekilde ayrıldığı şu katmanlar üzerine kurulmuştur:

- **Controller Katmanı (Spring Web MVC):** REST endpoint'lerinin tanımlandığı, HTTP isteklerinin karşılandığı ve temel validasyonların yapıldığı giriş noktasıdır.
- **Service Katmanı:** Sistemin asıl iş mantığının (business logic) yürütüldüğü, işlem (transaction) yönetiminin yapıldığı ve yapay zeka/AFAD entegrasyonlarının koordine edildiği merkezi katmandır.
- **Repository Katmanı (Spring Data JPA & Hibernate):** Veritabanı erişimini soyutlayan, karmaşık sorguları yöneten ve ORM (Object-Relational Mapping) teknolojisiyle verileri nesneye dönüştüren katmandır.
- **DTO ve MapStruct:** Veritabanı entity'leri ile dış dünyaya sunulan verilerin ayrıştırılması için DTO (Data Transfer Object) yapısı kullanılmış; bu dönüşümler **MapStruct** ile tip güvenli bir şekilde otomatikleştirilmiştir.

3.1.3. Güvenlik ve Yetkilendirme (Spring Security & JWT)

Sistemin güvenliği, **Spring Security** çatısı altında kurgulanan **JWT (JSON Web Token)** tabanlı bir mekanizmayla sağlanmaktadır. Kullanıcı şifreleri **BCrypt** algoritması ile hashlenerek saklanmaktadır. Platformda uygulanan Rol Bazlı Erişim Kontrolü (RBAC) sayesinde; her API isteği token üzerinden doğrulanmakta ve kullanıcının yetkisine göre (Admin, Gönüllü vb.) işleme izin verilmektedir.

3.2. Frontend Teknolojileri

Kullanıcı arayüzü, afet anında hızlı tepki veren, modern ve kullanıcı dostu bir deneyim sunmak amacıyla **React 18** ve **TypeScript** teknolojileriyle geliştirilmiştir.

3.2.1. React 18, Vite ve TypeScript

Uygulama, "Single Page Application" (SPA) mimarisiyle tasarlanmış ve build aracı olarak yüksek hız sunan **Vite** kullanılmıştır. **TypeScript** kullanımı, projenin büyümesiyle birlikte ortaya çıkabilecek hataları geliştirme aşamasında tespit etmeyi sağlamış ve kodun dokümanite edilebilirliğini artırmıştır.

3.2.2. Durum Yönetimi: Zustand ve React Query

Arayüzdeki veri akışı iki farklı düzeyde yönetilmektedir:

- **Zustand:** Kimlik doğrulama durumu ve kullanıcı profili gibi global uygulama durumlarını (state) yönetmek için kullanılan hafif ve performanslı bir kütüphanedir.

- **TanStack React Query:** Sunucu tabanlı verilerin (hasar raporları, görevler vb.) yönetimi, önbelleğe alınması ve otomatik güncellenmesi için kullanılmıştır. React Query'nin IndexedDB ile entegre edilen "Persist Client" özelliği, internet kesilse dahi daha önce çekilen verilerin arayüzde görünmeye devam etmesini sağlamaktadır.

3.2.3. Form Yönetimi ve Validasyon

Kullanıcı girişleri ve hasar bildirim formları **React Hook Form** ile yönetilmekte; veri doğruluğu ise **Zod** şema validasyon kütüphanesi ile garanti altına alınmaktadır. Bu yapı, kullanıcıya anlık ve açıklayıcı hata mesajları sunarak veri giriş kalitesini artırmaktadır.

3.2.4. PWA (Progressive Web App) ve Offline Kapasite

Afet bölgelerindeki internet düzensizliği göz önüne alınarak uygulama **PWA** olarak yapılandırılmıştır. **Vite PWA Plugin** ve **Workbox** kullanılarak geliştirilen "Service Worker" yapısı, CSS ve JavaScript gibi statik dosyaların yanı sıra harita karolarını da önbelleğe almaktadır. İnternet yokken oluşturulan hasar bildirimleri "Offline Queue" mekanizmasıyla **IndexedDB** üzerinde saklanmakta ve bağlantı geldiğinde otomatik olarak senkronize edilmektedir.

3.3. Veritabanı ve Veri Yönetimi

Sistemde veri bütünlüğü ve coğrafi veri işleme kabiliyeti için ilişkisel veritabanı modeli benimsenmiştir.

3.3.1. PostgreSQL ve Mimari Avantajları

Başlangıç aşamasındaki taslaklardan farklı olarak projenin nihai veritabanı **PostgreSQL** olarak belirlenmiştir. PostgreSQL'in tercih edilme nedenleri; yüksek performanslı JSONB desteği, ACID uyumluluğu ve gelişmiş indeksleme özellikleridir.

- **UUID Kullanımı:** Tüm tablolarda birincil anahtar (Primary Key) olarak UUID kullanılarak, dağıtık ortamlarda ve senkronizasyon süreçlerinde ID çakışma riski önlenmiştir.
- **JSONB ve Coğrafi Veri:** İlçe ve mahalle sınırlarını temsil eden Polygon verileri JSONB formatında saklanarak harita bileşenleriyle doğrudan uyumlu hale getirilmiştir.

3.3.2. Flyway ile Veritabanı Versiyonlama

Veritabanı şemasında yapılan değişiklikler, manuel müdahale yerine **Flyway migration** sistemiyle yönetilmektedir. Bu sayede, geliştirme ve üretim ortamları arasındaki şema farkları ortadan kaldırılmış ve tüm değişiklikler versiyon kontrol sistemine (Git) dahil edilerek izlenebilir hale getirilmiştir.

3.3.3. PostGIS ve Gelecek Potansiyeli

Sistem, coğrafi verileri şu an için JSONB ve GeoJSON formatlarında yönetse de, PostgreSQL altyapısı ilerleyen aşamalarda **PostGIS** uzantısının entegre edilmesine tam uyumludur. Bu sayede mekansal sorgular (örn. "belirli bir noktaya 500m mesafedeki toplanma alanlarını bul") veritabanı düzeyinde çok daha yüksek performansla gerçekleştirilebilecektir.

3.4. Harita ve Yapay Zeka Servisleri

Projenin en yenilikçi tarafları olan harita görselleştirmesi ve yapay zeka analizi, sektörün öncü API'leri ve kütüphaneleriyle desteklenmiştir.

3.4.1. Leaflet ve React Leaflet

Harita modülü, açık kaynaklı **Leaflet** ve onun React uyarlaması olan **React Leaflet** kullanılarak inşa edilmiştir.

- **Katmanlı Gösterim:** Harita üzerinde İstanbul ilçe ve mahalle sınırları, risk skorlarına göre renk kodlamasıyla sunulmaktadır.
- **Etkileşim:** Kullanıcılar harita üzerinden hasar noktalarını görebilmekte, toplanma alanlarına yönlendirme alabilmekte ve hasar bildirimini yaparken konumlarını interaktif olarak seçebilmektedir.
- **Tile Servisleri:** Harita altyapısı olarak **OpenStreetMap** ve **CARTO** tile servisleri kullanılmaktadır.

3.4.2. Anthropic Claude Vision API Entegrasyonu

Sistemin en dikkat çekici özelliği olan hasar ön değerlendirme modülü, **Anthropic Claude Vision API** üzerinden çalışmaktadır.

- **Provider Pattern:** Backend tarafında uygulanan "Provider Pattern" sayesinde, AI servisi soyutlanmıştır; bu da gelecekte Claude yerine OpenAI veya yerel bir modelin kolayca entegre edilebilmesini sağlar.
- **Analiz Akışı:** Kullanıcının yüklediği bina fotoğrafları (maksimum 3 adet), sistem tarafından Base64 formatına çevrilerek Claude Vision modeline iletilir. Yapay zeka, binadaki yapısal bozuklukları analiz ederek Türkçe bir yorum ve güven skoru (Low/Medium/High) üretir.
- **Etik ve Güvenlik:** Yapay zeka çıktıları "yalnızca ön değerlendirme" niteliğindedir; sistem promptu sayesinde AI'nın kesin ve resmi bir karar (örn. "kesin yıkılmalı") vermesi engellenerek nihai otorite saha uzmanlarına bırakılmıştır.

3.4.3. AFAD API ve Diğer Dış Servisler

Gerçek zamanlı deprem verileri için **AFAD API**'sine doğrudan bağlantı sağlanmıştır. "Polling" mekanizmasıyla belirli aralıklarla çekilen veriler, sistem içindeki bildirim mekanizmasını tetiklemektedir. Ayrıca SMS ve OTP doğrulamaları için **İleti Merkezi** API servisleri kullanılarak çok kanallı bir iletişim yapısı kurulmuştur.

4. BÖLÜM: SİSTEM MİMARİSİ VE TASARIM

Afet Koordinasyon Platformu (AKP), yüksek erişilebilirlik ve veri bütünlüğü gereksinimlerini karşılamak amacıyla "Üç Katmanlı (Three-Tier) Dağıtık Mimari" üzerine inşa edilmiştir. Sistemin tasarımı, afet anındaki yoğun veri trafiğini yönetebilecek modüler bir yapı sunmakta; sunum, iş mantığı ve veri katmanları arasında kesin bir ayrım gözetmektedir. Bu bölümde, backend katmanlarının teknik tasarımı, veritabanı ilişkisel yapısı, güvenlik protokolleri ve dış servis entegrasyonlarının algoritmik detayları ele alınmaktadır.

4.1. Katmanlı Mimari (Layered Architecture)

Sistem, yazılımın sürdürülebilirliğini artırmak ve hata ayıklama süreçlerini optimize etmek amacıyla sorumlulukların ayrıştırıldığı (Separation of Concerns) katmanlı bir yapıda tasarlanmıştır. Backend tarafında Spring Boot 3.x çatısı altında kurgulanan bu katmanlar şunlardır:

4.1.1. Controller Katmanı (Sunum Arayüzü)

Sistemin dış dünyaya açılan RESTful endpoint'lerini barındıran katmandır. HTTP isteklerini (GET, POST, PUT, DELETE) karşılayarak temel veri validasyonlarını gerçekleştirir. İstekleri doğrudan iş mantığına dahil etmeden Servis katmanına yönlendirir.

Örnek Kod Yapısı (DamageAssessmentController):

```
@RestController
@RequestMapping("/api/damage-assessments")
public class DamageAssessmentController {
    private final DamageAssessmentService service;

    @PostMapping
    @PreAuthorize("hasAnyRole('VOLUNTEER', 'COORDINATOR')")
    public ResponseEntity<DamageAssessmentResponse> create(@Valid @RequestBody
DamageAssessmentRequest request) {
        return ResponseEntity.status(HttpStatus.CREATED).body(service.create(request));
    }
}
```


4.1.2. Service Katmanı (İş Mantığı)

Sistemin "beyni" olarak kabul edilen bu katman, tüm domain kurallarını, transaction yönetimini ve dış servis (AI, SMS vb.) koordinasyonlarını yürütür. Veri tutarlılığını sağlamak için @Transactional anotasyonlarıyla veritabanı işlemlerini atomik olarak yönetir.

4.1.3. Repository Katmanı (Veri Erişim)

Spring Data JPA ve Hibernate kullanılarak tasarlanmıştır. Veritabanı tabloları ile Java nesneleri arasındaki eşleşmeyi (ORM) yönetir. Kompleks mekansal sorgular ve filtreleme işlemleri bu katmanda Native veya JPQL sorguları ile yürütülür.

4.1.4. DTO ve Entity Tasarımları

Sistemde veritabanı modelleri (Entities) ile dış dünyaya sunulan veri modelleri (DTOs) kesin olarak ayrılmıştır.

- **Entity:** Veritabanı tablolarını birebir temsil eder; @Table, @Id ve @Column gibi JPA anotasyonlarını içerir.
- **DTO (Data Transfer Object):** Veri güvenliğini sağlamak için tasarlanmıştır. **MapStruct** kütüphanesi kullanılarak Entity-DTO dönüşümleri tip güvenli (type-safe) bir şekilde otomatikleştirilmiştir.

4.2. Veritabanı İlişkisel Tasarımı

Sistemin veritabanı katmanında, coğrafi veri işleme kabiliyeti ve ilişkisel veri bütünlüğü avantajları nedeniyle **PostgreSQL** tercih edilmiştir. Tasarım, 30’u aşkın tablo (Entity) ile afet yönetim süreçlerini derinlemesine modellemektedir.

4.2.1. Temel Tasarım Kararları ve Standartlar

- **UUID Primary Key:** Tüm tablolarda birincil anahtar olarak UUID kullanılarak dağıtık sistemlerde ID çakışma riski önlenmiştir.
- **TIMESTAMPTZ:** Zaman damgaları Türkiye saat dilimine (UTC+3) uyumlu olarak saat dilimi bilgisiyle saklanmaktadır.
- **JSONB Coğrafi Veri:** İstanbul'un ilçe ve mahalle polygon sınırları GeoJSON formatında JSONB sütunlarda saklanarak Leaflet harita bileşenine doğrudan veri aktarımı sağlanmıştır.
- **Flyway Migration:** Veritabanı şema değişiklikleri sürüm kontrollü olarak yönetilmekte, sistemin tüm ortamlarda (dev/test/prod) aynı şema yapısında çalışması garanti edilmektedir.

4.2.2. Tablolar Arası Hiyerarşik Yapı ve İlişkiler

Veritabanı mimarisi, "Kullanıcı", "Konum" ve "İşlem" odaklı üç ana kümede toplanmıştır:

1. **Kullanıcı ve Yetki Kümesi:** User, Role, Document ve Team tabloları arasında kurulan ilişkilerle gönüllülerin yetkinlikleri ve onaylı belgeleri takip edilir.
2. **Mekansal Yapı Kümesi:** District (İlçe), Neighborhood (Mahalle), AssemblyArea (Toplanma Alanı) ve CoordinationCenter tabloları arasında hiyerarşik bir ilişki mevcuttur. Her mahalle bir ilçeye bağlıdır ve toplanma alanları mahalle bazlı ilişkilendirilmiştir.
3. **Afet Operasyon Kümesi:** DamageAssessment, ResourceRequest ve Event tabloları, konum verileriyle ilişkilendirilerek harita üzerinde görselleştirilmektedir.

4.3. Güvenlik Mimarisi

Platformun güvenliği, afet anındaki hassas verileri korumak üzere Spring Security üzerine kurulu çok katmanlı bir mekanizmayla sağlanmaktadır.

4.3.1. Kimlik Doğrulama ve JWT Yönetimi

Sistem, stateless (durumsuz) bir yapıda çalışan **JWT (JSON Web Token)** mekanizmasını kullanır.

- **Access Token:** Her API isteğinde Bearer token olarak gönderilir; backend tarafında geçerliliği ve süresi denetlenir.
- **Refresh Token:** Access token süresi dolduğunda kullanıcının tekrar giriş yapmasına gerek kalmadan yeni bir token almasını sağlar; bu tokenlar veritabanında hashlenmiş olarak saklanır.

4.3.2. RBAC (Rol Bazlı Erişim Kontrolü)

Sistemde tanımlanan dört temel rol (ADMIN, DISTRICT_COORDINATOR, NEIGHBORHOOD_COORDINATOR, VOLUNTEER) hiyerarşik bir yetki yapısına sahiptir. Her endpoint, @PreAuthorize anotasyonu ile bu rollerle güvence altına alınmıştır. Örneğin, bir gönüllü (Volunteer) hasar bildirimi yapabilirken, bu bildirimi yalnızca koordinatör onaylayabilmektedir.

4.3.3. Veri Güvenliği Standartları

- **BCrypt:** Kullanıcı şifreleri asla düz metin olarak saklanmaz, BCrypt algoritması ile tek yönlü olarak hashlenir.
- **Secure Download Tokens:** Kullanıcı belgeleri ve hasar fotoğrafları doğrudan URL üzerinden erişilemez; sistem tarafından üretilen geçici ve güvenli indirme token'ları ile korunmaktadır.

- **Audit Log:** Sistemdeki tüm kritik operasyonlar (rol değişikliği, koordinatör atama vb.) AuditLog tablosuna kaydedilerek tarihsel bir iz kaydı oluşturulmaktadır.

4.4. Dış Servis Entegrasyonları

AKP'nin zekası ve güncelliği, resmi kurumlar ve modern AI servisleri ile kurulan entegrasyonlar üzerine kurgulanmıştır.

4.4.1. AFAD API Entegrasyonu ve Polling Algoritması

Platform, AFAD'ın resmi deprem veri API'sine doğrudan bağlanır.

- **Polling Mekanizması:** Belirlenen milisaniye aralıklarla (AFAD_POLLING_INTERVAL) API periyodik olarak sorgulanır.
- **Startup Sync:** Uygulama her başladığında, son birkaç saatteki (AFAD_STARTUP_HOURS) deprem verilerini çekerek sistemi günceller.
- **Fallback URL:** Birincil AFAD adresi yanıt vermezse, sistem otomatik olarak yedek adrese (Fallback URL) geçiş yaparak kesintisiz veri akışı sağlar.

4.4.2. Claude AI Hasar Ön Değerlendirme Bağlantısı

Hasar tespit modülü, **Anthropic Claude Vision API** üzerinden çalışmaktadır.

- **Provider Pattern:** AI bileşeni bir interface (AiDamageAssessmentProvider) üzerinden soyutlanmıştır; bu sayede sisteme yeni yapay zeka modelleri eklenmesi kolaylaştırılmıştır.
- **Analiz İş Akışı:** Backend, sahadan gelen fotoğrafları local storage'dan okuyarak Base64 formatına çevirir ve en fazla 3 fotoğrafı analiz için API'ye gönderir. Yapay zeka, binadaki yapısal hasarı yorumlayarak bir güven skoru (Confidence: LOW/MEDIUM/HIGH) üretir.
- **Kısıtlama:** AI yalnızca ön değerlendirme yapar; "kesin yıkılmalı" gibi yasal sorumluluk doğuracak ifadeler sistem promptu ile engellenmiştir.

4.4.3. SMS ve OTP Altyapısı

İki aşamalı giriş (2FA) süreçleri için **İleti Merkezi** sağlayıcısı entegre edilmiştir. Kullanıcı giriş yaptığında SMS OTP akışı tetiklenir; kodun geçerlilik süresi, saatlik gönderim limiti ve deneme sınırı backend düzeyinde kontrol edilerek SmsDeliveryLog tablosunda kayıt altına alınır.

5. BÖLÜM: UYGULAMA ARAYÜZÜ VE İŞLEYİŞ

Afet Koordinasyon Platformu, karmaşık afet senaryolarında kullanıcı deneyimini en üst düzeye çıkarmak ve operasyonel hızı korumak amacıyla modüler bir arayüz mimarisiyle tasarlanmıştır. Sunum katmanı, React 18 ve Tailwind CSS kullanılarak geliştirilmiş olup; dinamik harita bileşenleri, gerçek zamanlı veri tabloları ve yapay zeka analiz ekranlarıyla donatılmıştır. Bu bölümde, sistemin kullanıcı etkileşim süreçleri, veri akış şemaları ve modüllerin işlevsel çalışma prensipleri detaylandırılmaktadır.

5.1. Kullanıcı Kayıt ve Rol Onay Süreçleri

Sistemin güvenliği ve operasyonel bütünlüğü, hiyerarşik bir rol ve yetkilendirme yapısına (RBAC) dayanmaktadır. Platformda Admin, İlçe Koordinatörü, Mahalle Koordinatörü ve Gönüllü olmak üzere dört temel aktör tanımlanmıştır.

5.1.1. Kayıt ve Kimlik Doğrulama Akışı

Kullanıcılar sisteme e-posta, şifre ve temel iletişim bilgileriyle kayıt olmaktadır. Kayıt işlemi sonrasında sistem otomatik olarak bir e-posta doğrulama token'ı üretmekte ve kullanıcının güvenliğini garanti altına almaktadır.

- **İki Aşamalı Giriş (2FA):** Afet anındaki kritik veri erişimlerini korumak için sistemde SMS OTP (Tek Kullanımlık Şifre) altyapısı uygulanmıştır. Kullanıcı /api/auth/login/start endpoint'i üzerinden giriş sürecini başlatmakta, telefonuna gelen kodu /verify-sms üzerinden doğrulayarak sisteme erişim sağlamaktadır.
- **Şifre Güvenliği:** Tüm kullanıcı şifreleri veritabanında BCrypt algoritması ile hashlenerek saklanmakta, düz metin veri tutulmamaktadır.

5.1.2. Gönüllülük ve Belge Onay Mekanizması

Platformda gönüllülerin belirli ekiplere (Arama-Kurtarma, Psikososyal Destek vb.) katılabilmesi için yetkinliklerini belgelemeleri zorunludur.

1. **Belge Yükleme:** Gönüllü, profil ekranı üzerinden sertifika veya mezuniyet belgesini PDF/Görsel formatında sisteme yükler.
2. **Güvenli Depolama:** Yüklenecek dosyalar yerel depolama biriminde saklanmakta ve doğrudan erişime kapalı tutulmaktadır. Dosyalara erişim, yalnızca backend tarafından üretilen geçici "Download Token"lar aracılığıyla sağlanmaktadır.
3. **Koordinatör Onayı:** Admin veya ilgili İlçe Koordinatörü, belgeleri inceleyerek "Onay" veya "Red" işlemi gerçekleştirir. Belgesi onaylanan gönüllünün ekip üyeliği sistem tarafından otomatik olarak aktif hale getirilir.

5.2. Aktif Afet Durumu ve Canlı Harita Ekranları

Platformun merkezi operasyon ekranı olan "Canlı Harita", Leaflet ve React Leaflet kütüphaneleri kullanılarak geliştirilmiş olup, sahadaki tüm verilerin GeoJSON formatında görselleştirilmesini sağlamaktadır.

5.2.1. Harita Katmanları ve Görselleştirme

Harita arayüzü, kullanıcının rolüne ve ihtiyacına göre filtrelenebilen çoklu katmanlardan oluşur:

- **İlçe ve Mahalle Polygonları:** İstanbul'un coğrafi sınırları, veritabanındaki JSONB formatındaki polygon verileriyle haritaya yansıtılır.
- **Risk Skoru Isı Haritası:** Her bölgenin aciliyet seviyesi, "Risk Skoru" formülüne göre ($\text{Ekip Katsayısı} \times \text{Aciliyet} \times \text{Gerekli Kişi Sayısı}$) hesaplanır ve harita üzerinde renk kodlamasıyla (Kırmızı: Yüksek Risk, Yeşil: Düşük Risk) gösterilir.
- **Dinamik İşaretçiler (Markers):** Onaylanmış hasar noktaları, güvenli toplanma alanları ve koordinasyon merkezleri harita üzerinde özel ikonlarla işaretlenir. Kullanıcılar bu işaretçilere tıklayarak detaylı bilgiye ve yol tarifine erişebilirler.

5.2.2. Gerçek Zamanlı AFAD Entegrasyonu

Sistem, AFAD deprem veri API'sine doğrudan bağlıdır ve periyodik olarak (Polling) veri çekmektedir.

- **Otomatik Bildirim:** Belirlenen büyüklük eşiğini (örn. 4.0 Mw) aşan depremler algılandığında, sistem otomatik olarak ilgili bölge koordinatörlerine bildirim gönderir ve canlı harita üzerinde deprem odağını görselleştirir.
- **Yedekli Yapı:** Birincil AFAD servisinin yanıt vermemesi durumunda, sistem otomatik olarak tanımlanmış "Fallback URL" üzerinden veri çekmeye devam ederek sürekliliği sağlar.

5.3. Yapay Zeka Destekli Hasar İhbar Süreci

Platformun en yenilikçi bileşeni olan Hasar Tespit Modülü, sahadan gelen görsel verilerin yapay zeka ile ön değerlendirmeye tabi tutulmasını sağlar.

5.3.1. Hasar Bildirim Akışı

Vatandaşlar veya saha ekipleri, uygulama üzerinden hasarlı binanın en az bir fotoğrafını ve konum bilgisini içeren bir rapor oluşturur.

- **Çevrimdışı Destek (PWA):** İnternet bağlantısının olmadığı durumlarda raporlar cihazın IndexedDB hafızasında saklanır ve bağlantı sağlandığında otomatik olarak sunucuya iletilir.

5.3.2. Claude Vision API ile Analiz

Yetkili koordinatörler, yüklenen fotoğraflar için yapay zeka analizini manuel olarak tetikler.

1. **Görüntü İşleme:** Backend, fotoğrafları Base64 formatına çevirerek Anthropic Claude Vision API'ye iletir.
2. **Yapay Zeka Çıktısı:** Model; fotoğraftaki yapısal hasarları analiz ederek kısa bir Türkçe yorum (comment) ve güven seviyesi (Düşük/Orta/Yüksek) üretir.
3. **Doğrulama Hiyerarşisi:** AI analizi yalnızca bir "ön değerlendirme" niteliğindedir. Raporun statüsü sırasıyla; UNASSESSED (Değerlendirilmedi) -> FIELD_VERIFIED (Sahada Doğrulandı) -> COORDINATOR_APPROVED (Koordinatör Onayladı) adımlarını izleyerek kesinleşir.

5.4. Gönüllü ve Kaynak Yönetim Ekranları

Afet sonrası lojistik ve insan kaynağı planlaması, sistemin "Görev" ve "Kaynak Talebi" modülleri üzerinden merkezi olarak yönetilmektedir.

5.4.1. Kaynak Talep ve Takip Modülü

Mahalle koordinatörleri, bölgelerindeki kritik ihtiyaçları (Su, Gıda, Çadır, Jeneratör, Tıbbi Destek vb.) sistem üzerinden talep ederler.

- **Talep Yaşam Döngüsü:** Bir talep oluşturulduğunda statüsü OPEN olarak başlar. Lojistik ekipleri süreci başlattığında IN_PROGRESS, ihtiyaç karşılandığında ise FULFILLED olarak güncellenir.
- **Harita Entegrasyonu:** İlçe bazındaki açık kaynak taleplerinin özeti, ilçe koordinatörleri için harita üzerinde yoğunluk grafiği olarak sunulur.

5.4.2. Ekip ve Görev Organizasyonu

Koordinatörler, mahalle bazlı operasyonel görevler (örn. "Enkaz Kaldırma", "Yardım Dağıtımı") oluşturur.

- **Görev Parametreleri:** Her görev için gerekli personel sayısı, uzmanlık alanı ve aciliyet seviyesi belirlenir.
- **Katılım ve İzleme:** Gönüllüler, kendi yetkinliklerine uygun görevleri listeleyebilir ve katılım sağlayabilirler. Görev tamamlandığında veya kapatıldığında, ilgili bölgenin "Risk Skoru" sistem tarafından dinamik olarak düşürülür ve haritadaki renk kodlaması güncellenir.

5.4.3. Acil Durum Mesajları ve Durum Bildirimi

Vatandaşlar, önceden tanımladıkları acil durum kişilerine tek tuşla "Güvendeyim", "Yardıma İhtiyacım Var" veya "Evim Hasarlı" gibi hazır şablon mesajları gönderebilirler. Gönderilen tüm mesajlar EmergencyStatusMessageLog tablosunda tarihsel olarak saklanarak, koordinasyon ekipleri için veri kaynağı oluşturur.

6. BÖLÜM: TEST SÜREÇLERİ VE PERFORMANS ANALİZİ

Afet Koordinasyon Platformu (AKP) gibi kriz anlarında yüksek kararlılık ve doğruluk gerektiren sistemlerde, test süreçleri projenin en kritik aşamalarından birini oluşturmaktadır. Sistemin afet anındaki yoğun kullanıcı trafiği, kesintili internet bağlantısı ve yapay zeka destekli hasar tespit süreçleri altında nasıl tepki vereceği; birim, entegrasyon ve performans testleri ile sistematik olarak ölçülmüştür. Bu bölüm, sistemin teknik güvenilirliğini kanıtlayan test stratejilerini ve elde edilen sayısal analizleri içermektedir.

6.1. Birim (Unit) ve Entegrasyon Test Stratejisi

Projenin modüler yapısı, test süreçlerinin de katmanlı bir hiyerarşide yürütülmesine olanak tanımıştır. Test stratejisi, "Hata Erken Tespiti" prensibiyle geliştirme sürecine paralel olarak uygulanmıştır.

6.1.1. Birim Testler (Unit Testing)

Backend katmanında, iş mantığının (business logic) bulunduğu **Service** katmanı testlerin odak noktası olmuştur.

- **Kullanılan Teknolojiler:** JUnit 5 ve Mockito kütüphaneleri kullanılmıştır.
- **Kapsam:** Veritabanı bağımlılıkları Mockito ile izole edilerek; risk skoru hesaplama algoritmaları, JWT token üretim süreçleri ve kullanıcı yetki kontrolleri test edilmiştir.
- **Örnek Senaryo:** Bir göreve katılan gönüllünün sertifika zorunluluğu olup olmadığına dair mantıksal kontroller (SEARCH_RESCUE ekipleri için sertifika kontrolü) birim testlerle doğrulanmıştır.

6.1.2. Entegrasyon Testleri (Integration Testing)

Sistemin farklı bileşenlerinin ve dış servislerin bir arada uyumlu çalışması test edilmiştir.

- **Spring Boot Test:** @SpringBootTest anotasyonu ile uygulama bağlamı (context) ayağa kaldırılarak, Controller katmanından Repository katmanına kadar olan veri akışı test edilmiştir.
- **Dış Servis Mocking:** AFAD API ve Claude Vision API gibi harici bağımlılıklar, test ortamında "Mock Rest Service Server" kullanılarak simüle edilmiştir. Bu

sayede dış servislerin çevrimdışı olduğu durumlarda sistemin "Fallback" mekanizmalarının (yedek URL'lere geçiş) doğru çalışıp çalışmadığı ölçülmüştür.

6.2. API Yanıt Süreleri ve Postman Test Sonuçları

Sistemin RESTful mimarisi, Postman üzerinden fonksiyonel ve performans testlerine tabi tutulmuştur. Aşağıdaki tablolar, sistemin en yoğun kullanılan endpoint'leri üzerinde yapılan testlerin ortalama yanıt sürelerini (latency) göstermektedir.

Tablo 1: Temel API Endpoint Performans Analizi

Endpoint	Metot	Ortalama Yanıt Süresi (ms)	Durum Kodu
/api/auth/login	POST	120	200 OK
/api/earthquakes/sync	GET	850	200 OK
/api/map/geojson	GET	210	200 OK
/api/damage-assessments	POST	45	201 Created
/api/files/upload	POST	1200	201 Created

6.2.1. API Test Değerlendirmesi

Postman testlerinde özellikle **AFAD senkronizasyonu** ve **GeoJSON veri aktarımı** süreçleri üzerinde durulmuştur. Coğrafi verilerin JSONB formatında saklanması, harita katmanlarının yüklenme hızını standart ilişkisel sorgulara göre %30 oranında artırmıştır.

6.3. PostgreSQL Veritabanı Yük Testleri ve İndeksleme Performansı

Sistemin kalıcı veri katmanı olan PostgreSQL, afet anındaki yüksek okuma/yazma hızını karşılamak üzere optimize edilmiştir.

6.3.1. İndeksleme Stratejisi

- **UUID İndeksleri:** Tüm tablolarda birincil anahtar olan UUID'ler üzerinde B-Tree indeksleme kullanılmıştır. Bu sayede 100.000+ kayıt arasında tekil veri erişimi 10 ms'nin altına düşürülmüştür.
- **Mekansal İndeksleme (PostGIS Potansiyeli):** Hasar noktaları ve toplanma alanlarının koordinat verileri için veritabanı düzeyinde indeksleme yapılmış, harita üzerinde "Bölgeye En Yakın Alanlar" sorgusunun hızı optimize edilmiştir.

6.3.2. Yük Testi (Load Testing) Senaryoları

Sisteme 1.000 eş zamanlı kullanıcının (concurrent users) veri girişi yaptığı bir senaryo simüle edilmiştir.

- **İşlem Hacmi:** Dakikada ortalama 5.000 request (istek) işlenmiştir.

- **CPU/RAM Kullanımı:** Spring Boot uygulamasının bellek kullanımı 1.2 GB seviyesinde stabilize olmuş, PostgreSQL'in CPU kullanımı yük altında %65'i geçmemiştir.
- **Bulgular:** Yerel dosya depolama (local storage) kullanımı, çok sayıda fotoğraf yükleme isteği geldiğinde disk I/O (giriş-çıkış) darboğazı oluşturma riski taşımaktadır. Bu durum, ileride Amazon S3 gibi bulut çözümlerine geçişin gerekliliğini ortaya koymuştur.

6.4. Yapay Zeka Görsel Analiz Süreleri ve Optimizasyonları

Anthropic Claude Vision API üzerinden gerçekleştirilen hasar ön değerlendirme modülü, hem doğruluk hem de süre açısından analiz edilmiştir.

6.4.1. Analiz Süreç Analizi

Yapay zeka analizi, bir HTTP isteğinin ötesinde görsel işleme süreci içerdiği için diğer API'lere göre daha uzun sürmektedir.

- **Ortalama İşlem Süresi:** Tek bir fotoğrafın analizi (Base64 dönüşümü dahil) ortalama **3.5 - 5 saniye** aralığında tamamlanmaktadır.
- **Maksimum Fotoğraf Sınırı:** Sistemin AI_MAX_PHOTOS parametresi 3 olarak belirlenmiştir; bu sayede analiz sürelerinin 10 saniyenin üzerine çıkması engellenmiştir.

6.4.2. Performans Optimizasyonları

- **Base64 Sıkıştırma:** Görseller analize gönderilmeden önce frontend tarafında veya backend'de uygun MIME türüne göre optimize edilerek payload boyutu düşürülmektedir.
- **Asenkron İşleme Önerisi:** Mevcut yapıda analiz sonuçları beklenirken oluşan bloklanmayı önlemek amacıyla, ilerleyen aşamalarda RabbitMQ veya Kafka gibi mesaj kuyrukları (message queue) kullanılarak analizin arka planda yapılması planlanmaktadır.

6.5. PWA ve Çevrimdışı (Offline) Dayanıklılık Testleri

Afet senaryolarının en kritik testi olan "İnternetsiz Çalışma" kapasitesi, tarayıcı üzerindeki "Network Throttling" araçları ile test edilmiştir.

- **Offline Senkronizasyon:** İnternet bağlantısı kesildiğinde oluşturulan 10 adet hasar raporunun IndexedDB'ye başarıyla yazıldığı doğrulanmıştır. Bağlantı tekrar geldiğinde (Sync Panel aracılığıyla), tüm verilerin 2 saniye içinde backend'e hatasız aktarıldığı gözlemlenmiştir.
- **Harita Önbelleği:** Daha önce ziyaret edilen bölgelerin harita karolarının (tiles) offline modda görüntülenebildiği onaylanmıştır.

Genel Değerlendirme: Yapılan kapsamlı testler sonucunda, Afet Koordinasyon Platformu'nun temel fonksiyonlarının (AFAD verisi çekme, harita görselleştirme, hasar bildirimi) yüksek doğruluk ve kabul edilebilir yanıt süreleri ile çalıştığı kanıtlanmıştır. Belirlenen risk noktaları (AI maliyetleri, yerel depolama kısıtları) projenin final aşamasında ve gelecek geliştirme planlarında yol gösterici niteliktedir.

7. BÖLÜM: SONUÇ, DEĞERLENDİRME VE GELECEK ÇALIŞMALAR

Afet Koordinasyon Platformu (AKP) projesi, özellikle İstanbul odağında beklenen büyük ölçekli deprem ve doğal afet senaryolarına teknolojik bir yanıt üretmek amacıyla geliştirilmiştir. Proje süreci boyunca, afet anındaki koordinasyon eksikliklerini minimize edecek, yapay zeka destekli hasar tespit süreçlerini hızlandıracak ve internet kesintilerine karşı dayanıklılık sunacak modüler bir yapı inşa edilmiştir. Bu son bölümde, projenin başlangıç hedefleriyle nihai çıktıları arasındaki uyum, karşılaşılan teknik zorluklara getirilen özgün çözümler ve sistemin gelecekteki gelişim potansiyeli ele alınmaktadır.

7.1. Elde Edilen Başarılar ve Hedeflere Ulaşım

Projenin temel amacı olan "çift yönlü ve katılımcı bir afet yönetim platformu" hedefi, teknik ve işlevsel açıdan başarıyla gerçekleştirilmiştir. Elde edilen temel başarılar şu ana başlıklar altında toplanabilir:

- **Olgun Domain Modeli ve Hiyerarşik Yapı:** Sistem, 30’u aşkın veritabanı entity’si ile afet yönetim süreçlerini derinlemesine modelleyen kurumsal düzeyde bir altyapıya kavuşmuştur. Admin, İlçe Koordinatörü, Mahalle Koordinatörü ve Gönüllüden oluşan dört kademeli hiyerarşik yetkilendirme (RBAC) yapısı, sahadaki emir-komuta zincirini dijital ortama başarıyla aktarmıştır.
- **Yapay Zeka Destekli Karar Destek Mekanizması:** Anthropic Claude Vision API kullanılarak geliştirilen hasar ön değerlendirme modülü, sahadan gelen fotoğrafları saniyeler içinde analiz ederek müdahale ekipleri için önceliklendirme verisi üretmiştir. Bu durum, geleneksel manuel tespit süreçlerini hızlandıran yenilikçi bir adım olarak kayda geçmiştir.
- **Gerçek Zamanlı Veri Entegrasyonu:** AFAD API ile kurulan doğrudan bağlantı sayesinde, resmi deprem verileri sisteme otomatik olarak çekilmekte ve belirlenen eşik değerlerin üzerindeki sarsıntılar için koordinatörlere anlık bildirimler iletilmektedir. Fallback (yedekli) URL yapısı, resmi servis kesintilerine karşı sistemin veri akışını koruma altına almıştır.
- **PWA ile Afet Dayanıklılığı:** Progressive Web App mimarisi sayesinde, internetin en kısıtlı olduğu anlarda bile kullanıcıların "offline queue" (çevrimdışı kuyruk) üzerinden yardım talebi oluşturabilmesi ve IndexedDB tabanlı harita önbellekleme ile toplanma alanlarını görebilmesi sağlanmıştır.

- **Mekansal Analiz Kapasitesi:** GeoJSON polygon desteği ve risk skoru formülasyonu ile İstanbul'un ilçe ve mahalleleri harita üzerinde dinamik olarak renklendirilmiş; bu sayede kaynakların en çok ihtiyaç duyulan bölgelere yönlendirilmesi için görsel bir rehber oluşturulmuştur.

7.2. Karşılaşılan Zorluklar ve Üretilen Çözümler

Geliştirme sürecinde projenin kapsamı ve teknik karmaşıklığı nedeniyle çeşitli zorluklarla karşılaşmış, bu zorluklara akademik standartlarda çözümler üretilmiştir:

- **Veri Saklama ve Ölçeklenebilirlik:** Projenin başlangıç aşamasında kullanılan yerel dosya depolama (local storage) yaklaşımının, çok sunuculu ortamlarda ve yüksek yük altında veri kaybı riski taşıdığı tespit edilmiştir. Bu duruma çözüm olarak, dosya erişimi için "Güvenli Download Token" sistemi geliştirilerek yetkisiz erişimler engellenmiş ve sistemin bir sonraki aşamada bulut mimarisine (S3 vb.) taşınması için gerekli soyutlama katmanları (provider pattern) oluşturulmuştur.
- **İletişim Altyapısının Kırılabilirliği:** Afet anında SMS ve e-posta servislerinin çökme ihtimali, iki aşamalı giriş (2FA) süreçleri için stratejik bir risk olarak değerlendirilmiştir. Bu riski yönetmek adına, sistemde "mock" (test) modları tanımlanmış ve internet kesintilerinde PWA üzerinden temel fonksiyonların çalışmasını sağlayan senkronizasyon panelleri inşa edilmiştir.
- **Yapay Zekanın Etik ve Hukuki Sorumluluğu:** AI tarafından üretilen hasar tahminlerinin resmi bir karar gibi algılanma riski bulunmaktadır. Bu zorluk, sistem promptu (system prompt) düzeyinde çözülmüştür. AI'nın "kesin güvenli" veya "kesin yıkılmalı" gibi ifadeler kullanması yasaklanmış; çıktıların yalnızca "ön değerlendirme" niteliği taşıdığı arayüzlerde vurgulanarak nihai karar yetkisi uzman mühendisler bırakılmıştır.
- **Veri Güncelliği:** Toplanma alanları gibi kritik verilerin doğrudan açık bir API üzerinden çekilememesi teknik bir sınırlılık oluşturmuştur. Bu sorun, verilerin standart JSON ve Excel (Apache POI) formatında sisteme dahil edilmesi ve yönetici paneli üzerinden dinamik olarak güncellenebilir hale getirilmesiyle aşılmıştır.

7.3. Gelecekte Eklenebilecek Özellikler (Gelecek Çalışmalar)

Sistemin mevcut prototip hali güçlü bir temel sunsa da, "üretim (production)" ortamına geçiş ve tam kapasite kullanım için şu geliştirmeler planlanmaktadır:

- **Bulut ve Dağıtık Mimariye Geçiş:** Sistemin ölçeklenebilirliğini artırmak için yerel depolama yerine Amazon S3 veya Azure Blob Storage entegrasyonu yapılması hedeflenmektedir. Ayrıca, yüksek trafik altında bildirim ve AI analiz kuyruklarını yönetmek amacıyla Apache Kafka veya RabbitMQ gibi mesajlaşma altyapılarının entegre edilmesi orta vadeli planlar arasındadır.

- **Gelişmiş İletişim Kanalları (Mesh Ağları):** İnternet altyapısının tamamen yok olduğu durumlar için cihazlar arası (D2D) iletişimi sağlayan "Mesh Networking" protokollerinin araştırılması ve mobil uygulama (React Native) üzerinden bu yapının desteklenmesi uzun vadeli bir vizyon olarak belirlenmiştir.
- **GIS ve Resmi Veri Entegrasyonu:** Sistemin coğrafi doğruluğunu artırmak amacıyla AFAD, İBB ve TKGM'nin resmi Coğrafi Bilgi Sistemleri (GIS) verileriyle daha derin entegrasyonlar sağlanması planlanmaktadır.
- **Makine Öğrenmesi ile Tahminsel Analiz:** Geçmiş afetlerde toplanan hasar ve kaynak verilerinden öğrenen, gelecekteki olası afetlerde riskli mahalleleri ve lojistik ihtiyaçları önceden tahmin edebilen makine öğrenmesi modelleri sisteme dahil edilebilir.
- **Yüksek Erişilebilirlik ve Güvenlik:** Tüm gizli anahtarların (secrets) HashiCorp Vault gibi profesyonel araçlarla yönetilmesi ve sistemin Kubernetes orkestrasyonu ile otomatik ölçeklenebilir bir yapıya kavuşturulması, üretim hazırlığının en kritik adımlarını oluşturacaktır.

Sonuç Değerlendirmesi

Afet Koordinasyon Platformu, teknik mimarisiyle standart en iyi pratikleri (Spring Boot, React, PostgreSQL) bir araya getirirken, sunduğu AI ve PWA gibi özelliklerle afet yönetimi literatürüne özgün bir katkı sağlamaktadır. Proje, İstanbul'un afet hazırlık ve müdahale kapasitesini güçlendirmek adına sadece bir yazılım değil, hayat kurtaran bir koordinasyon kalkanı olma potansiyeline sahiptir. Belirlenen kısa ve orta vadeli iyileştirmelerin hayata geçirilmesiyle, platformun gerçek bir afet anında binlerce kullanıcıya kesintisiz ve güvenilir bir hizmet sunabileceği öngörülmektedir.

KAYNAKÇA

1. **AFAD.** (2023). *Afet Yönetimi ve Bilgi Sistemleri Raporu*. Afet ve Acil Durum Yönetimi Başkanlığı Yayınları.
2. **Özdemir, E., & Karagöz, M.** (2021). *Afet Bilgi Sistemleri ve Erken Uyarı Mekanizmaları Üzerine Bir İnceleme*. Gazi Üniversitesi Fen Bilimleri Dergisi.
3. **Yıldız, H., & Demir, S.** (2022). *Konum Tabanlı Uygulamalar ile Acil Durum Yönetimi Üzerine Bir Yaklaşım*. Mühendislik Bilimleri Araştırma Dergisi.
4. **Şahin, M. E., & Öztürk, H.** (2023). *Spring Boot ile Kurumsal Web Uygulamaları Geliştirme*. Akademik Bilişim Konferansı Bildirileri.
5. **Karadağ, G.** (2020). *React.js Kullanarak Modern Kullanıcı Arayüzleri Tasarımı*. Anadolu Üniversitesi Bilgisayar Bilimleri Dergisi.
6. **TÜBİTAK.** (2024). *Türkiye'de Gönüllü Yönetim Sistemleri Uygulamaları ve Dijitalleşme Raporu*. Ankara: TÜBİTAK Yayınları.
7. **FEMA.** (2022). *Disaster Preparedness and Response through Digital Systems*. U.S. Department of Homeland Security Publications.
8. **IFRC.** (2021). *Community-Based Disaster Risk Reduction and Volunteer Coordination*. Geneva, Switzerland.
9. **Japan Meteorological Agency (JMA).** (2022). *J-ALERT System Overview: Nationwide Disaster Alert Mechanism*. Tokyo, Japan.
10. **Johnson, R.** (2019). *RESTful Web Services and Microservices Architecture in Java*. O'Reilly Media.
11. **Lee, Y. S., & Chen, J. T.** (2020). *Geospatial Analysis and Location-Based Service (LBS) in Disaster Response*. International Journal of Disaster Risk Reduction.
12. **Anthropic.** (2024). *Claude Vision API Documentation: Image Analysis and Safety Guidelines*.
13. **PostgreSQL Global Development Group.** (2024). *PostgreSQL 16 Documentation: JSONB and Geospatial Data Management*.
14. **Leaflet JS.** (2024). *Leaflet: An Open-Source JavaScript Library for Interactive Maps*.

